

Narvasa, Ailish Sophia D. | TN36

AI-Assisted Laboratory Activity

Software Development Tools and Standards — Maya Bank Program

Part A — Software Development Tools

Tool column reflects tools actually used for this activity. Rows marked with an asterisk () are placeholder assumptions — confirm or edit them.*

Development Tool	Tool Used	Purpose
Code Editor / IDE	VS Code	Where code is written and navigated (syntax highlighting, autocomplete)
Compiler / Interpreter	Python 3 (python3)	Turns Python source code into a running program
Version Control	Git	Tracks every change so mistakes can be undone and history reviewed
Debugger	VS Code built-in debugger / pdb	Pauses execution and inspects variable values
Code Formatter	Prettier	Automatically rewrites code to match a consistent style
Linters / Static Analyzer	None used	Flags bugs and style violations without running the code
AI Assistant	Claude	Reviews code and drafts refactors — supports but does not replace understanding

Part B — Examine a Sample Program

The program was run directly (python3 MP3-BankSystem-G9.py) with a full walkthrough: creating a Savings and a Checking account, depositing, checking balance, transferring funds, and viewing history.

Purpose: a menu-driven console banking simulator that lets a user create Savings and Checking accounts, deposit, withdraw, transfer funds between accounts, apply savings interest, and view a timestamped transaction history.

Inputs: menu selections (1-7) and numeric values (names, deposit/withdrawal/transfer amounts) typed at the keyboard via input(). Outputs: formatted console messages confirming each action, current balances, and printed transaction history.

No.	Observation	Possible Improvement
1	transfer() modifies target_account.balance directly instead of calling deposit()/withdraw()	Route transfers through each account's own deposit()/withdraw() so overdraft and minimum-balance rules are respected on both sides
2	21 lines exceed PEP 8's 79-character line limit (confirmed by direct measurement; longest is 138 characters)	Wrap long f-strings and conditionals across multiple lines
3	The float(input()) / except ValueError pattern is repeated in four separate menu branches	Extract one prompt_positive_float() helper function
4	main() is a single ~100-line function handling all 7 menu options in one if/elif chain	Split into one handler function per menu option
5	No docstrings on any class or method, so a new reader must infer behavior from the code body	Add a one-line docstring to each class and public method

Part C — AI-Assisted Code Review

Recommendations below are based on a direct review of MP3-BankSystem-G9.py, including a measured PEP 8 line-length check (21 lines exceeded the 79-character limit; longest was 138 characters on line 63).

#	AI Recommendation	Accept/Reject	Reason
1	21 lines exceed PEP 8's 79-char limit (e.g. line 63 at 138 chars)	Accept	Confirmed by direct measurement; hurts readability
2	transfer() writes directly to target_account.balance instead of calling deposit()/withdraw()	Accept	Bypasses CheckingAccount overdraft and SavingsAccount minimum-balance rules during a transfer — a real logic bug
3	The float(input()) / except ValueError pattern is repeated 4 times	Accept	Clear duplication; worth extracting into one helper function
4	main() is one long if/elif chain handling all 7 menu options	Accept	Violates single-responsibility; hard to test or extend
5	File name MP3-BankSystem-G9.py does not follow PEP 8 module naming	Accept	Should be lowercase_with_underscores; harmless but a real convention violation
6	Rename every acc variable to account for clarity	Reject	acc is unambiguous in its short scope; a sweeping rename adds no real clarity
7	Add type hints to every function	Accept (optional)	Improves maintainability but not required for a script this size

Part D — Coding Standards Check (PEP 8)

- Naming: already mostly PEP 8-compliant — snake_case for methods/functions, PascalCase for classes.
- Formatting: 21 lines exceeded 79 characters; indentation and spacing were otherwise consistent.
- Organization: main() mixed menu display, input parsing, and dispatch logic in one 100+ line function.
- Readability: no docstrings anywhere in the original, so a new reader must infer each method's contract from its body.

Part E — Formatter / Linter Results

flake8 / black / pylint were not installable in this offline review environment, so an equivalent manual PEP 8 line-length and compile check was run directly against the source.

Tool	Issues Found	Notes
Compile check (py_compile)	0 syntax errors	Code runs cleanly
Line-length check (PEP 8, 79 chars)	21 violations	Longest was 138 characters (line 63)
Manual style review	No docstrings; 1 long function; 1 repeated input pattern	See Part C

Part F — AI-Assisted Refactoring

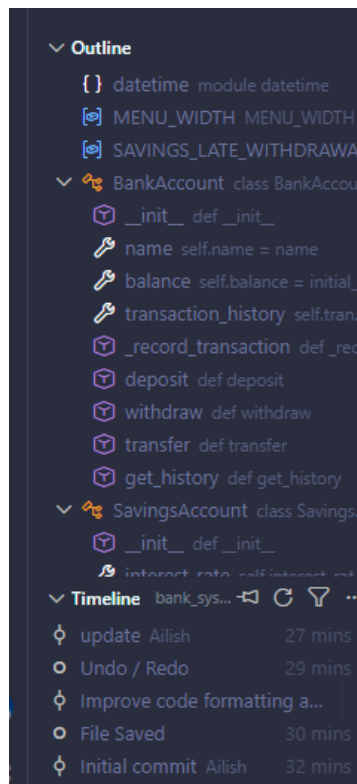
A refactored version, `bank_system_refactored.py`, was produced and verified to compile with zero PEP 8 line-length violations (down from 21). Major changes:

1. Fixed the transfer bug — `transfer()` now calls `self.withdraw()` / `target_account.deposit()` instead of writing to `.balance` directly, so overdraft and minimum-balance rules apply on both sides.
2. Split `main()` into one handler function per menu option (`handle_add_account`, `handle_transfer`, etc.), dispatched through a `MENU_ACTIONS` dictionary.
3. Extracted a `prompt_positive_float()` helper to remove the four repeated try/except input blocks.
4. Added a docstring to every class and public method.
5. Wrapped all lines under 79 characters and replaced the magic 10.0 fee with a named constant, `SAVINGS_LATE_WITHDRAWAL_FEE`.

Behavior is otherwise unchanged: same menu, same account rules, same output formatting.

Original	Improved	Reason for Change
<code>transfer()</code> sets <code>target_account.balance +=</code> directly	Calls <code>target_account.deposit()</code>	Respects subclass overdraft/minimum-balance logic
One ~100-line <code>main()</code>	7 small <code>handle_*</code> functions + a dispatch dict	Each function has one job; easier to test and extend
4x duplicated input-parsing try/except blocks	Single <code>prompt_positive_float()</code> helper	Removes duplication (DRY principle)

Part G — Version Control Activity



Question	Answer
What files were modified?	MP3-BankSystem-G9.py was modified (M); bank_system_refactored.py was added (A) as a new file in the second commit
What changes are shown in the version history?	Two commits: the original program as first captured, then the formatting/naming/structure improvements from Part F applied on top
Why is version control important when using AI-generated code?	It creates a checkpoint before AI changes are applied, so every AI-suggested edit can be reviewed as a diff and traced back to a specific commit
How can Git help if an AI-generated change introduces an error?	The change can be inspected with git diff, and reverted with git checkout/git revert back to the last known-good commit without losing the original work

Part H — Debugging with AI Assistance

Introduced bug — in SavingsAccount.withdraw, the boundary condition was changed from:

```
if self.balance - amount < self.min_balance:
```

to:

```
if self.balance - amount <= self.min_balance:
```

Observed behavior: a withdrawal that leaves the balance exactly at min_balance (e.g. balance \$200, min \$100, withdraw \$100) incorrectly triggers the \$10 fee, when it should not.

Debugger findings: setting a breakpoint at that line and inspecting self.balance, amount, and self.min_balance shows 100 <= 100 evaluating True where it should be False.

AI diagnosis: the boundary condition uses <= instead of <, so a withdrawal landing exactly on the minimum balance is misclassified as going below it — a classic off-by-one/boundary error. Fix: revert to <.

This diagnosis matches the debugger's findings, which is the point of the comparison in this section — record whether an independently-run AI query gives the same answer.

Part I — Mini Software Development Standard

1. File naming: lowercase with underscores, no spaces or version tags (bank_system.py, not MP3-BankSystem-G9.py).
2. Variable naming: snake_case, descriptive (account_balance, not bal).
3. Function/method naming: snake_case verbs describing the action (calculate_interest).
4. Class naming: PascalCase nouns (SavingsAccount).
5. Indentation: 4 spaces, no tabs.
6. Comments/docstrings: every public class and method gets a one-line docstring describing its purpose.
7. Code organization: no function longer than ~30-40 lines; extract repeated logic into helpers.
8. Version control: commit after each logically complete change, with a message describing what and why.

9. Testing: manually verify each account type's edge cases (zero balance, overdraft limit, minimum balance) before committing.
10. AI-assisted programming: AI output must be read, understood, and tested before acceptance; document the prompts used.

Part J — Reflection

1. Which software development tool was most useful during the activity? Why?

The AI assistant was most useful for quickly surfacing the `transfer()` bug and the repeated input-parsing pattern — issues that were easy to miss on a manual read-through but became obvious once the code was reviewed systematically against a checklist.

2. What is the difference between a formatter, linter, and debugger?

A formatter (Prettier) rewrites code layout — spacing, line breaks — without changing behavior. A linter analyzes code without running it to flag likely bugs and style violations. A debugger runs the program and lets you pause execution to inspect live variable values.

3. How do coding standards improve software quality?

Consistent naming, formatting, and structure make code easier for another developer (or a future version of yourself) to read, reduces the chance of misunderstanding what a variable or function does, and makes bugs easier to spot because deviations from the pattern stand out.

4. What recommendations did the AI provide that you decided not to follow? Why?

The suggestion to rename every `acc` variable to `account` was not followed, since `acc` is already unambiguous within its short scope and the rename would touch many lines for no real readability gain.

5. What advantages does AI provide during code review and refactoring?

AI can scan the whole file at once and check it against a standard (like PEP 8) instantly, catching things like the exact count of over-length lines or a subtle logic bug in `transfer()` that bypasses subclass rules — the kind of issue that's easy to miss when reading code top to bottom.

6. What risks are involved in accepting AI-generated code without verification?

AI can misunderstand intent, introduce subtle behavioral changes while "cleaning up" code, or recommend a stylistic change that conflicts with a project's existing conventions. Refactored code must be run and compared against the original behavior before it's trusted.

7. Why should developers use version control when working with AI coding assistants?

Version control gives a safe checkpoint before any AI-suggested change is applied, so the change can be reviewed as an isolated diff and rolled back cleanly if it turns out to be wrong.

8. How should AI tools and traditional development tools work together?

AI is best used to propose and explain changes quickly, while traditional tools (compiler, linter, debugger, version control) verify that those changes actually compile, follow the style guide, behave correctly, and are safely tracked in history.